

# OPMP 核心函式修改指南 - 五階段關鍵函式索引

版本: v1.0.0 日期: 2025-10-03 作者: Claude (SuperClaude) 系統: SalesRAG Progressive Streaming System

## 目錄

- [1. 文檔使用說明](#)
- [2. Phase 1: Query Understanding](#)
- [3. Phase 2: Parallel Retrieval](#)
- [4. Phase 3: Context Assembly](#)
- [5. Phase 4: Response Generation](#)
- [6. Phase 5: Post-processing](#)
- [7. 跨階段共享函式](#)
- [8. 修改影響分析矩陣](#)

## 文檔使用說明

### 修改風險等級說明

| 等級  | 符號 | 說明                 | 建議                    |
|-----|----|--------------------|-----------------------|
| 低風險 |    | 局部函式,影響範圍小,有容錯機制   | 可直接修改,做好單元測試          |
| 中風險 |    | 核心邏輯函式,影響單一階段      | 修改前備份,做好整合測試          |
| 高風險 |    | 關鍵路徑函式,影響多階段或系統穩定性 | 必須 Code Review,完整回歸測試 |

### 函式表格格式

每個函式都包含以下資訊: - **函式名稱**: 完整函式簽章 - **位置**: 檔案路徑:行號 - **責任**: 函式職責描述 - **輸入/輸出**: 參數和返回值 - **修改風險**: / / - **影響範圍**: 修改後波及的其他組件

## Phase 1: Query Understanding & Entity Extraction

檔案: `libs/services/sales_assistant/phase1_query_understanding.py` 階段職責: 分析用戶查詢,提取意圖、產品型號、關鍵特徵

### 1.1 主流程函式

`async def process(query, available_modelnames, available_modeltypes)`

| 屬性   | 詳情                                                |
|------|---------------------------------------------------|
| 位置   | <code>phase1_query_understanding.py:78-179</code> |
| 風險等級 | 高風險 - Phase 1 入口點                                 |

| 屬性   | 詳情                                                                                                                                                   |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 職責   | Phase 1 主流程協調器,依序執行 cache → fast path → LLM → fallback                                                                                               |
| 輸入   | query: str - 用戶查詢<br>available_modelnames: List[str] - 可用產品型號<br>available_modeltypes: List[str] - 可用產品類別                                            |
| 輸出   | AsyncGenerator[Dict, None] - yield progress updates 和 phase_result                                                                                   |
| 關鍵邏輯 | 1. Cache check (line 104-122)<br>2. Fast path extraction (line 124-145)<br>3. LLM extraction (line 147-167)<br>4. Fallback extraction (line 169-179) |
| 修改影響 | • 影響 Phase 1 所有子流程<br>• Phase 2 依賴其輸出的 analysis                                                                                                      |
| 修改場景 | • 新增額外的分析路徑<br>• 調整流程優先級<br>• 新增資料驗證                                                                                                                 |

## 1.2 核心分析函式

```
def _fast_path_extraction(query, available_modelnames,
available_modeltypes)
```

| 屬性   | 詳情                                                                                                                                                                                            |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase1_query_understanding.py:181-262                                                                                                                                                         |
| 風險等級 | <b>中風險</b> - 影響簡單查詢的處理速度                                                                                                                                                                      |
| 職責   | 使用 Regex 快速提取產品資訊和意圖,適用於明確的查詢                                                                                                                                                                 |
| 輸入   | query: str<br>available_modelnames: List[str]<br>available_modeltypes: List[str]                                                                                                              |
| 輸出   | Optional[Dict] - 分析結果或 None                                                                                                                                                                   |
| 關鍵邏輯 | • Product pattern: \b(APX AHP AG ARB AMD AB AKK)\s*(\d{3,4})\b<br>• Modeltype pattern: \b(819 839 928 958 960 AC01)\b<br>• Intent keywords: 比較/推薦/規格查詢<br>• Feature keywords: CPU/GPU/記憶體/電池等 |
| 修改影響 | • 影響簡單查詢的命中率<br>• 不影響 LLM fallback 機制                                                                                                                                                         |
| 修改場景 | • 新增產品型號 pattern<br>• <b>擴充關鍵特徵識別</b> (如新增 "散熱" 關鍵詞)<br>• 調整 confidence 判斷邏輯                                                                                                                  |

**修改範例:** 擴充電池關鍵詞

# Before (line 227):

"電池": ["電池", "电池", "續航", "续航", "續航力", "续航力", "battery", "充電", "充电"]

# After - 新增 "待機" 和 "電量":

"電池": ["電池", "电池", "續航", "续航", "續航力", "续航力", "battery", "充電", "充电", "待機", "待机", "電量", "电量"]

```
async def _llm_extraction(query, available_modelnames,
available_modeltypes)
```

| 屬性   | 詳情                                                                                                                                                                                               |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase1_query_understanding.py:264-310                                                                                                                                                            |
| 風險等級 | 中風險 - LLM 調用核心邏輯                                                                                                                                                                                 |
| 職責   | 使用 LLM 進行深度查詢分析,處理複雜或模糊的查詢                                                                                                                                                                       |
| 輸入   | query: str<br>available_modelnames: List[str]<br>available_modeltypes: List[str]                                                                                                                 |
| 輸出   | Dict - 分析結果 (包含 intent, detected_products 等)                                                                                                                                                     |
| 關鍵邏輯 | <ul style="list-style-type: none"> <li>• 使用 QUERY_UNDERSTANDING_PROMPT template (line 22-40)</li> <li>• 調用 self.llm.invoke(prompt)</li> <li>• JSON 解析 response</li> <li>• Fallback 處理</li> </ul> |
| 修改影響 | <ul style="list-style-type: none"> <li>• 影響 LLM 分析準確度</li> <li>• Token 消耗</li> </ul>                                                                                                             |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整 <b>Prompt template</b> 提升準確度</li> <li>• 新增額外的提取欄位</li> <li>• 更換 LLM 模型</li> </ul>                                                                    |

**修改範例:** Prompt template 調整

# 在 QUERY\_UNDERSTANDING\_PROMPT (line 22-40) 中新增額外指示:

```
QUERY_UNDERSTANDING_PROMPT = """你是一個專業的產品查詢分析助手。請分析以下用戶查詢並提取
```

```
用戶查詢：{user_query}
```

```
可用產品型號（部分）：{available_modelnames}
```

```
可用機型類別：{available_modeltypes}
```

請以 JSON 格式回答，包含以下欄位：

```
{{
  "intent": "compare|recommend|spec_query|general_inquiry",
  "detected_products": ["產品1", "產品2"],
  "detected_modeltypes": ["819", "839"],
  "key_features": ["CPU", "GPU", "記憶體"],
  "user_focus": "效能|價格|攜帶性|電池續航力",
  "complexity": "simple|medium|complex",
  "budget_mentioned": true|false, # 新增：是否提及預算
  "urgency": "low|medium|high"    # 新增：購買急迫性
}}
```

只回覆 JSON，不要其他說明。

"""

```
def _fallback_extraction(query)
```

| 屬性   | 詳情                                                                            |
|------|-------------------------------------------------------------------------------|
| 位置   | phase1_query_understanding.py:347-365                                         |
| 風險等級 | 低風險 - 容錯機制                                                                    |
| 職責   | 所有方法失敗時的保底分析,返回保守的預設值                                                         |
| 輸入   | query: str                                                                    |
| 輸出   | Dict - 基本分析結果                                                                 |
| 關鍵邏輯 | 返回保守預設值: intent="general_inquiry", complexity="medium" 等                      |
| 修改影響 | <ul style="list-style-type: none"><li>• 僅影響極端失敗場景</li><li>• 確保系統不中斷</li></ul> |
| 修改場景 | <ul style="list-style-type: none"><li>• 調整預設特徵清單</li><li>• 新增預設值欄位</li></ul>  |

### 1.3 快取管理函式

```
async def _get_cached_analysis(query) & async def  
_cache_analysis(query, analysis)
```

| 屬性   | 詳情                                                                                                           |
|------|--------------------------------------------------------------------------------------------------------------|
| 位置   | phase1_query_understanding.py:367-401                                                                        |
| 風險等級 | 低風險 - 獨立的快取層                                                                                                 |
| 職責   | Redis 快取的讀取和寫入                                                                                               |
| 輸入   | query: str, analysis: Dict (cache 時)                                                                         |
| 輸出   | Optional[Dict] (get 時), None (set 時)                                                                         |
| 關鍵邏輯 | <ul style="list-style-type: none"><li>• Cache key: phase1:{md5(query)}</li><li>• TTL: 300 秒 (5 分鐘)</li></ul> |
| 修改影響 | <ul style="list-style-type: none"><li>• 不影響核心邏輯</li><li>• 僅影響效能</li></ul>                                    |
| 修改場景 | <ul style="list-style-type: none"><li>• 調整 TTL</li><li>• 更換 cache key 生成策略</li><li>• 新增 cache 失效條件</li></ul> |

### Phase 1 修改總結

最常見修改場景: 1. 擴充特徵識別 → 修改 `_fast_path_extraction()` 的 `feature_keywords` 2. 提升 LLM 準確度 → 調整 `QUERY_UNDERSTANDING_PROMPT` 3. 新增分析欄位 → 修改 `_llm_extraction()` 和 `_fallback_extraction()`

修改注意事項: - ⚠ 修改 Prompt template 後需測試 JSON 解析穩定性 - ⚠ 新增欄位需同步更新 Phase 2/3 的使用邏輯 - ✓ `_fast_path_extraction()` 修改風險較低,可快速迭代

## Phase 2: Parallel Multi-source Data Retrieval

檔案: `libs/services/sales_assistant/phase2_parallel_retrieval.py` 階段職責:  
並行從 Milvus 和 DuckDB 檢索資料,合併去重

### 2.1 主流程函式

`async def retrieve(query, detected_products, top_k, use_cache)`

| 屬性   | 詳情                                                                                                                                                                  |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase2_parallel_retrieval.py:141-270</code>                                                                                                                   |
| 風險等級 | 高風險 - Phase 2 入口點                                                                                                                                                   |
| 職責   | Phase 2 主流程,協調並行檢索和結果合併                                                                                                                                             |
| 輸入   | <code>query: str</code><br><code>detected_products: Optional[List[str]]</code> - 從 Phase 1 來<br><code>top_k: int = 30</code><br><code>use_cache: bool = True</code> |
| 輸出   | <code>AsyncGenerator[Dict, None]</code> - yield progress 和 phase_result                                                                                             |
| 關鍵邏輯 | 1. Cache check (line 171-190)<br>2. Parallel retrieval (line 201-204)<br>3. Result merging (line 222)<br>4. Cache result (line 239-240)                             |
| 修改影響 | • 影響所有資料檢索邏輯<br>• Phase 3 依賴其輸出的 <code>retrieval_results</code>                                                                                                     |
| 修改場景 | • 調整並行策略<br>• 新增第三資料源<br>• 修改 <code>top_k</code> 預設值                                                                                                                |

### 2.2 並行檢索核心函式

`async def _parallel_retrieve(query, detected_products, top_k)`

| 屬性   | 詳情                                                                                                                     |
|------|------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase2_parallel_retrieval.py:271-314</code>                                                                      |
| 風險等級 | 高風險 - 並行執行核心                                                                                                           |
| 職責   | 使用 <code>asyncio.gather()</code> 並行執行 Milvus 和 DuckDB 查詢                                                               |
| 輸入   | <code>query: str</code><br><code>detected_products: Optional[List[str]]</code><br><code>top_k: int</code>              |
| 輸出   | <code>tuple[List[Dict], List[Dict]]</code> - (semantic_results, spec_results)                                          |
| 關鍵邏輯 | • 建立兩個 <code>async tasks</code><br>• <code>asyncio.gather(*tasks, return_exceptions=True)</code><br>• 錯誤處理: 單一失敗不影響另一個 |
| 修改影響 | • 影響檢索效能<br>• 影響錯誤處理策略                                                                                                 |
| 修改場景 | • 新增第三資料源 (如 Elasticsearch)                                                                                            |

| 屬性 | 詳情                 |
|----|--------------------|
|    | • 調整並行策略 (如改為順序執行) |

**修改範例:** 新增 Elasticsearch 資料源

# 修改 \_parallel\_retrieve() (line 271-314):

```

async def _parallel_retrieve(
    self,
    query: str,
    detected_products: Optional[List[str]],
    top_k: int
) -> tuple[List[Dict], List[Dict], List[Dict]]: # 新增第三個返回值
    """Execute Milvus, DuckDB, and Elasticsearch queries in parallel"""

    self.stats['parallel_retrievals'] += 1

    # Create tasks for parallel execution
    tasks = [
        self._retrieve_from_milvus(query, top_k),
        self._retrieve_from_duckdb(detected_products, query, top_k),
        self._retrieve_from_elasticsearch(query, top_k) # 新增
    ]

    # Execute in parallel
    results = await asyncio.gather(*tasks, return_exceptions=True)

    # Handle results
    semantic_results = results[0] if not isinstance(results[0], Exception) else None
    spec_results = results[1] if not isinstance(results[1], Exception) else []
    es_results = results[2] if not isinstance(results[2], Exception) else [] #

    # Log any errors
    if isinstance(results[0], Exception):
        logger.error(f"Milvus retrieval error: {results[0]}")
    if isinstance(results[1], Exception):
        logger.error(f"DuckDB retrieval error: {results[1]}")
    if isinstance(results[2], Exception):
        logger.error(f"Elasticsearch retrieval error: {results[2]}") # 新增

    return semantic_results, spec_results, es_results # 修改返回值

```

**async def \_retrieve\_from\_milvus(query, top\_k)**

| 屬性   | 詳情                                   |
|------|--------------------------------------|
| 位置   | phase2_parallel_retrieval.py:316-381 |
| 風險等級 | 中風險 - Milvus 查詢邏輯                    |
| 職責   | 從 Milvus 向量資料庫檢索語義相似的產品片段            |
| 輸入   | query: str<br>top_k: int             |

| 屬性   | 詳情                                                                                                                                                                                             |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 輸出   | List[Dict] - 語義匹配結果                                                                                                                                                                            |
| 關鍵邏輯 | <ul style="list-style-type: none"> <li>• 使用 Sentence Transformer 生成 embedding</li> <li>• Milvus 向量搜尋 (L2 距離)</li> <li>• 計算 similarity_score: <math>1 / (1 + \text{distance})</math></li> </ul> |
| 修改影響 | <ul style="list-style-type: none"> <li>• 影響語義搜尋準確度</li> <li>• 不影響其他資料源</li> </ul>                                                                                                              |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整 <b>search_params</b> (如 nprobe)</li> <li>• 更換 embedding 模型</li> <li>• 調整相似度計算公式</li> </ul>                                                         |

**修改範例:** 調整搜尋參數提升準確度

# 修改 search\_params (line 340-343):

# Before:

```
search_params = {
    "metric_type": "L2",
    "params": {"nprobe": 10}
}
```

# After - 提升 nprobe 增加搜尋範圍:

```
search_params = {
    "metric_type": "L2",
    "params": {"nprobe": 20} # 10 → 20, 更全面但稍慢
}
```

**async def \_retrieve\_from\_duckdb(detected\_products, query, top\_k)**

| 屬性   | 詳情                                                                                                                                                            |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase2_parallel_retrieval.py:383-419                                                                                                                          |
| 風險等級 | 中風險 - DuckDB 查詢邏輯                                                                                                                                             |
| 職責   | 從 DuckDB 檢索結構化產品規格資料                                                                                                                                          |
| 輸入   | detected_products: Optional[List[str]]<br>query: str<br>top_k: int                                                                                            |
| 輸出   | List[Dict] - 產品規格資料                                                                                                                                           |
| 關鍵邏輯 | <ul style="list-style-type: none"> <li>• 如有 detected_products → query_by_modeltypes()</li> <li>• 否則 → 基本 SELECT 查詢</li> <li>• 只查詢 ESSENTIAL_FIELDS</li> </ul> |
| 修改影響 | <ul style="list-style-type: none"> <li>• 影響規格資料完整性</li> <li>• 影響 I/O 效能</li> </ul>                                                                            |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整 <b>ESSENTIAL_FIELDS</b> (新增/移除欄位)</li> <li>• 優化 fallback 查詢邏輯</li> </ul>                                          |

**修改範例:** 擴充 ESSENTIAL\_FIELDS

```
# 修改 ESSENTIAL_FIELDS (line 74-78):

# Before:
ESSENTIAL_FIELDS = [
    'modeltype', 'modelname', 'cpu', 'gpu', 'memory', 'storage',
    'lcd', 'battery', 'audio', 'wireless', 'bluetooth',
    'softwareconfig', 'thermal', 'ai'
]

# After - 新增 'warranty' 和 'price':
ESSENTIAL_FIELDS = [
    'modeltype', 'modelname', 'cpu', 'gpu', 'memory', 'storage',
    'lcd', 'battery', 'audio', 'wireless', 'bluetooth',
    'softwareconfig', 'thermal', 'ai',
    'warranty', 'price' # 新增欄位
]
```

## 2.3 結果處理函式

```
def _merge_results(semantic_results, spec_results)
```

| 屬性   | 詳情                                                                                                                                           |
|------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase2_parallel_retrieval.py:421-492                                                                                                         |
| 風險等級 | 高風險 - 資料合併核心邏輯                                                                                                                               |
| 職責   | 合併 Milvus 語義結果和 DuckDB 規格資料,處理去重                                                                                                             |
| 輸入   | semantic_results: List[Dict]<br>spec_results: List[Dict]                                                                                     |
| 輸出   | List[Dict] - 合併且去重的產品列表                                                                                                                      |
| 關鍵邏輯 | <ul style="list-style-type: none"> <li>• 關鍵: 基於 modelname 去重,而非 modeltype</li> <li>• 合併語義分數和規格資料</li> <li>• 按 semantic_score 降序排序</li> </ul> |
| 修改影響 | <ul style="list-style-type: none"> <li>• 影響最終產品列表準確性</li> <li>• 影響 Phase 3 的資料品質</li> </ul>                                                  |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整去重策略</li> <li>• 修改排序邏輯</li> <li>• 新增資料驗證</li> </ul>                                               |

⚠ **重要:** 此函式已修復過 bug (多個產品共享同一 modeltype),修改時需特別注意去重邏輯。

## Phase 2 修改總結

**最常見修改場景:** 1. 調整檢索欄位 → 修改 ESSENTIAL\_FIELDS 2. 提升搜尋準確度 → 調整 `_retrieve_from_milvus()` 的 `search_params` 3. 新增資料源 → 修改 `_parallel_retrieve()` 和 `_merge_results()`

**修改注意事項:-** 關鍵: `_merge_results()` 的去重邏輯基於 modelname,不是 modeltype -



⚠ 新增 ESSENTIAL\_FIELDS 會影響 I/O 和 token 消耗 - ⚠ 調整並行策略需考慮錯誤處理機制 -  
✓ Milvus 和 DuckDB 查詢相對獨立,可分別優化

## Phase 3: Context Assembly & Ranking

檔案: `libs/services/sales_assistant/phase3_context_assembly.py` 階段職責: 產品排序、欄位篩選、token 管理

### 3.1 主流程函式

```
async def process(retrieval_results, analysis)
```

| 屬性   | 詳情                                                                                                                   |
|------|----------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase3_context_assembly.py:47-117</code>                                                                       |
| 風險等級 | 高風險 - Phase 3 入口點                                                                                                    |
| 職責   | Phase 3 主流程,協調排序和截斷                                                                                                  |
| 輸入   | <code>retrieval_results</code> : Dict - Phase 2 輸出<br><code>analysis</code> : Dict - Phase 1 輸出                      |
| 輸出   | <code>AsyncGenerator[Dict, None]</code> - yield progress 和 <code>phase_result</code>                                 |
| 關鍵邏輯 | 1. 檢查 <code>merged_products</code> (line 71-85)<br>2. Rank products (line 87-92)<br>3. Truncate context (line 94-99) |
| 修改影響 | • 影響所有排序和截斷邏輯<br>• Phase 4 依賴其輸出的 context                                                                            |
| 修改場景 | • 調整 token 限制<br>• 修改處理流程                                                                                            |

### 3.2 核心排序函式

```
def _rank_products_by_relevance(products, key_features, detected_products)
```

| 屬性   | 詳情                                                                                                                        |
|------|---------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase3_context_assembly.py:119-198</code>                                                                           |
| 風險等級 | 中風險 - 產品排序邏輯                                                                                                              |
| 職責   | 基於多重標準計算產品相關性分數並排序                                                                                                        |
| 輸入   | <code>products</code> : List[Dict]<br><code>key_features</code> : List[str]<br><code>detected_products</code> : List[str] |
| 輸出   | <code>List[Dict]</code> - 排序後的產品列表 (附加 <code>relevance_score</code> )                                                     |
| 關鍵邏輯 | 排序標準 (line 142-186):<br>1. 精確產品匹配: +100 分<br>2. 語義相似度: +0~50 分<br>3. 特徵完整性: +0~20 分<br>4. 產品新舊: +0~10 分                   |
| 修改影響 | • 影響產品顯示順序                                                                                                                |

| 屬性   | 詳情                                                                                                      |
|------|---------------------------------------------------------------------------------------------------------|
|      | <ul style="list-style-type: none"> <li>• 影響 LLM 優先參考哪些產品</li> </ul>                                     |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整排序權重</li> <li>• 新增排序標準 (如價格、庫存)</li> <li>• 修改特徵映射</li> </ul> |

#### 修改範例: 調整排序權重

# 修改權重 (line 145-173):

# Before:

```
if modelname in detected_products or modeltype in detected_products:
    score += 100.0 # 精確匹配
```

```
if "similarity_score" in product:
    score += product["similarity_score"] * 50.0 # 語義相似度
```

```
feature_completeness = feature_count / len(key_features) if key_features else 0
score += feature_completeness * 20.0 # 特徵完整性
```

# After - 提升特徵完整性權重:

```
if modelname in detected_products or modeltype in detected_products:
    score += 100.0 # 精確匹配 (不變)
```

```
if "similarity_score" in product:
    score += product["similarity_score"] * 40.0 # 語義相似度 50 → 40
```

```
feature_completeness = feature_count / len(key_features) if key_features else 0
score += feature_completeness * 30.0 # 特徵完整性 20 → 30 (提升)
```

### 3.3 Context 截斷函式

```
def _truncate_context(products, max_tokens, key_features)
```

| 屬性   | 詳情                                                                                                 |
|------|----------------------------------------------------------------------------------------------------|
| 位置   | phase3_context_assembly.py:200-290                                                                 |
| 風險等級 | 高風險 - Token 管理核心                                                                                   |
| 職責   | 智慧截斷產品列表和欄位以符合 token 限制                                                                            |
| 輸入   | products: List[Dict]<br>max_tokens: int<br>key_features: List[str]                                 |
| 輸出   | Dict - 包含 products, token_count, truncation_applied 等                                              |
| 關鍵邏輯 | 截斷策略 (line 223-282):<br>1. 保留 Top 10 產品<br>2. 選擇必要欄位 (預設 + 動態)<br>3. 提取必要欄位, 截斷語義內容<br>4. Token 估算 |

| 屬性   | 詳情                                                                                                                          |
|------|-----------------------------------------------------------------------------------------------------------------------------|
|      | 5. 超限時進一步截斷                                                                                                                 |
| 修改影響 | <ul style="list-style-type: none"> <li>• 影響 LLM 收到的資料完整性</li> <li>• 影響系統 token 消耗</li> </ul>                                |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整 <b>essential_fields</b></li> <li>• 修改 token 估算方法</li> <li>• 調整產品數量限制</li> </ul> |

**修改範例:** 調整必要欄位

# 修改 essential\_fields (line 227-229):

# Before:

```
essential_fields = ['modeltype', 'modelname', 'cpu', 'gpu', 'memory', 'storage']
```

# After - 新增 'warranty' 和 'price':

```
essential_fields = ['modeltype', 'modelname', 'cpu', 'gpu', 'memory', 'storage',
                    'battery', 'lcd', 'warranty', 'price']
```

---

**def \_estimate\_tokens(text)**

| 屬性   | 詳情                                                                                                              |
|------|-----------------------------------------------------------------------------------------------------------------|
| 位置   | phase3_context_assembly.py:292-316                                                                              |
| 風險等級 | 低風險 - Token 估算輔助函式                                                                                              |
| 職責   | 估算文本的 token 數量                                                                                                  |
| 輸入   | text: str                                                                                                       |
| 輸出   | int - 估算的 token 數                                                                                               |
| 關鍵邏輯 | <ul style="list-style-type: none"> <li>• 優先使用 tiktoken (GPT-4 encoding)</li> <li>• Fallback: 字元數 ÷ 3</li> </ul> |
| 修改影響 | <ul style="list-style-type: none"> <li>• 僅影響 token 估算準確度</li> <li>• 不影響核心邏輯</li> </ul>                          |
| 修改場景 | <ul style="list-style-type: none"> <li>• 更換 tiktoken encoding</li> <li>• 調整 fallback 公式</li> </ul>              |

---

## Phase 3 修改總結

**最常見修改場景:** 1. 調整保留欄位 → 修改 `_truncate_context()` 的 `essential_fields` 2. 優化排序邏輯 → 調整 `_rank_products_by_relevance()` 的權重 3. 修改 **token 限制** → 調整 `max_tokens` 參數

**修改注意事項:** - 關鍵: `essential_fields` 必須包含 `battery`, `lcd` 等常用欄位 - ⚠ 修改排序權重需考慮對產品優先順序的影響 - ⚠ 擴充 `essential_fields` 會增加 token 消耗 - ✓ Token 估算邏輯可獨立優化

---

## Phase 4: Response Generation (Progressive Markdown)

檔案: `libs/services/sales_assistant/phase4_response_generation.py` 階段職責: LLM 生成 Markdown 回應並進行 token-by-token 串流

### 4.1 主流程函式

`async def process(query, analysis, context)`

| 屬性   | 詳情                                                                                                                                                |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase4_response_generation.py:131-270</code>                                                                                                |
| 風險等級 | 高風險 - Phase 4 入口點                                                                                                                                 |
| 職責   | Phase 4 主流程, 協調 LLM 生成和串流輸出                                                                                                                       |
| 輸入   | query: str<br>analysis: Dict - Phase 1 輸出<br>context: Dict - Phase 3 輸出                                                                           |
| 輸出   | AsyncGenerator[Dict, None] - yield markdown tokens 和 progress                                                                                     |
| 關鍵邏輯 | 1. Cache check (line 157-176)<br>2. Build prompt (line 178-179)<br>3. LLM streaming generation (line 184-251)<br>4. Cache response (line 254-255) |
| 修改影響 | • 影響 LLM 生成品質<br>• Phase 5 依賴其輸出的 <code>generated_response</code>                                                                                 |
| 修改場景 | • 調整 LLM 參數<br>• 修改串流策略<br>• 更換 LLM 模型                                                                                                            |

### 4.2 Prompt 構建函式

`def _build_prompt(query, analysis, context)`

| 屬性   | 詳情                                                                                                                                                                                                        |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase4_response_generation.py:272-300</code>                                                                                                                                                        |
| 風險等級 | 中風險 - Prompt 構建邏輯                                                                                                                                                                                         |
| 職責   | 構建完整的 LLM prompt, 包含查詢、分析、產品資料                                                                                                                                                                            |
| 輸入   | query: str<br>analysis: Dict<br>context: Dict                                                                                                                                                             |
| 輸出   | str - 完整的 prompt                                                                                                                                                                                          |
| 關鍵邏輯 | • 使用 <code>RESPONSE_GENERATION_PROMPT</code> template (line 23-41)<br>• 格式化產品資訊 (line 290)<br>• 填充 <code>user_query</code> , <code>intent</code> , <code>user_focus</code> , <code>product_context</code> |
| 修改影響 | • 影響 LLM 生成品質和格式<br>• 影響 token 消耗                                                                                                                                                                         |
| 修改場景 | • 調整 <b>Prompt template</b> 提升回應品質<br>• 新增額外的 context 資訊                                                                                                                                                  |

修改範例: 強化 Prompt template

# 修改 RESPONSE\_GENERATION\_PROMPT (line 23-41):

# Before:

RESPONSE\_GENERATION\_PROMPT = """你是一個專業的筆記型電腦銷售助手。請根據以下產品資料回答

用戶查詢：{user\_query}

查詢分析：

- 意圖：{intent}
- 關注重點：{user\_focus}

產品資料：

{product\_context}

請以繁體中文回答，格式要求：

1. 使用 Markdown 格式 (headers, bold, tables)
2. 如果是產品比較，使用表格呈現關鍵規格差異
3. 提供清晰的購買建議，說明適用場景
4. 保持專業但友善的語氣

開始回答：

"""

# After - 新增強制引用資料的指示：

RESPONSE\_GENERATION\_PROMPT = """你是一個專業的筆記型電腦銷售助手。請根據以下產品資料回答

用戶查詢：{user\_query}

查詢分析：

- 意圖：{intent}
- 關注重點：{user\_focus}

產品資料：

{product\_context}

請以繁體中文回答，格式要求：

1. 使用 Markdown 格式 (headers, bold, tables)
2. 如果是產品比較，使用表格呈現關鍵規格差異
3. 提供清晰的購買建議，說明適用場景
4. 保持專業但友善的語氣
5. **\*\*重要\*\***：必須引用產品資料中的具體規格數據，不要憑空推測

特別注意：

- 如果產品資料中有電池容量（如 80Wh，99Wh），務必在回答中明確提及
- 所有規格描述必須基於提供的產品資料，不要使用模糊的描述

開始回答：

"""

```
def _format_product_context(products)
```

| 屬性   | 詳情                                                                                                                                         |
|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase4_response_generation.py:302-347                                                                                                      |
| 風險等級 | 中風險 - 產品資料格式化                                                                                                                              |
| 職責   | 將產品 dict 列表格式化為結構化文本                                                                                                                       |
| 輸入   | products: List[Dict]                                                                                                                       |
| 輸出   | str - 格式化的產品資訊文本                                                                                                                           |
| 關鍵邏輯 | <ul style="list-style-type: none"><li>• 為每個產品建立 Markdown 格式區塊</li><li>• 包含核心規格: CPU, GPU, 記憶體, 儲存, 螢幕, 電池</li><li>• 可選特徵: 散熱, AI</li></ul> |
| 修改影響 | <ul style="list-style-type: none"><li>• 影響 LLM 看到的產品資訊格式</li><li>• 影響 token 消耗</li></ul>                                                   |
| 修改場景 | <ul style="list-style-type: none"><li>• 調整顯示的規格欄位</li><li>• 修改格式化樣式</li></ul>                                                              |

## 4.3 快取管理函式

```
async def _get_cached_response(query, context) & async def  
_cache_response(...)
```

| 屬性   | 詳情                                                                                                                           |
|------|------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase4_response_generation.py:349-402                                                                                        |
| 風險等級 | 低風險 - 快取層                                                                                                                    |
| 職責   | Redis 快取的讀取和寫入                                                                                                               |
| 輸入   | query: str, context: Dict, response: str (cache 時)                                                                           |
| 輸出   | Optional[str] (get 時), None (set 時)                                                                                          |
| 關鍵邏輯 | <ul style="list-style-type: none"><li>• Cache key: phase4:{md5(query + product_ids)}</li><li>• TTL: 1800 秒 (30 分鐘)</li></ul> |
| 修改影響 | <ul style="list-style-type: none"><li>• 不影響核心邏輯</li><li>• 僅影響效能</li></ul>                                                    |
| 修改場景 | <ul style="list-style-type: none"><li>• 調整 TTL</li><li>• 修改 cache key 策略</li></ul>                                           |

## Phase 4 修改總結

最常見修改場景: 1. 提升回應品質 → 調整 RESPONSE\_GENERATION\_PROMPT 2. 修改顯示規格 → 調整 \_format\_product\_context() 的欄位 3. 更換 LLM 模型 → 修改 \_\_init\_\_() 的 LLM 初始化

修改注意事項: - 關鍵: Prompt template 必須明確要求 LLM 引用資料 - ⚠ 修改 Prompt 需測試不同查詢類型的回應 - ⚠ \_format\_product\_context() 必須包含所有核心規格 - ✓ 快取邏輯

可獨立調整 TTL

## Phase 5: Post-processing & Formatting

檔案: `libs/services/sales_assistant/phase5_postprocessing.py` 階段職責:  
Markdown 驗證、Metadata 新增、品質檢查

### 5.1 主流程函式

**`async def process(generated_response, context, analysis, query)`**

| 屬性   | 詳情                                                                                                                                                                            |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase5_postprocessing.py:50-123</code>                                                                                                                                  |
| 風險等級 | 中風險 - Phase 5 入口點                                                                                                                                                             |
| 職責   | Phase 5 主流程, 協調後處理和品質檢查                                                                                                                                                       |
| 輸入   | <code>generated_response</code> : str - Phase 4 輸出<br><code>context</code> : Dict - Phase 3 輸出<br><code>analysis</code> : Dict - Phase 1 輸出<br><code>query</code> : str       |
| 輸出   | AsyncGenerator[Dict, None] - yield complete response_package                                                                                                                  |
| 關鍵邏輯 | 1. Build metadata (line 78)<br>2. Generate sources (line 81)<br>3. Validate & fix markdown (line 84)<br>4. Quality check (line 96)<br>5. Build response_package (line 87-100) |
| 修改影響 | • 影響最終輸出格式<br>• 不影響前序階段                                                                                                                                                       |
| 修改場景 | • 新增 metadata 欄位<br>• 調整品質檢查標準                                                                                                                                                |

### 5.2 Metadata 與來源函式

**`def _build_metadata(context, analysis)`**

| 屬性   | 詳情                                                                                                                                           |
|------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | <code>phase5_postprocessing.py:125-150</code>                                                                                                |
| 風險等級 | 低風險 - Metadata 構建                                                                                                                            |
| 職責   | 構建回應的 metadata 資訊                                                                                                                            |
| 輸入   | <code>context</code> : Dict<br><code>analysis</code> : Dict                                                                                  |
| 輸出   | Dict - metadata                                                                                                                              |
| 關鍵邏輯 | 包含: <code>products_analyzed</code> , <code>context_tokens</code> , <code>query_intent</code> , <code>model</code> , <code>timestamp</code> 等 |
| 修改影響 | • 僅影響 metadata 內容<br>• 不影響核心功能                                                                                                               |
| 修改場景 | • 新增額外的統計資訊<br>• 新增系統資訊                                                                                                                      |

---

**def \_generate\_sources(context)**

| 屬性   | 詳情                                                                            |
|------|-------------------------------------------------------------------------------|
| 位置   | phase5_postprocessing.py:152-178                                              |
| 風險等級 | 低風險 - 來源引用生成                                                                  |
| 職責   | 為每個分析的產品生成來源引用                                                                |
| 輸入   | context: Dict                                                                 |
| 輸出   | List[Dict] - 來源清單                                                             |
| 關鍵邏輯 | 包含: product_id, product_name, source_type, relevance_score                    |
| 修改影響 | <ul style="list-style-type: none"><li>• 僅影響來源引用內容</li><li>• 不影響核心功能</li></ul> |
| 修改場景 | <ul style="list-style-type: none"><li>• 新增額外的來源資訊</li></ul>                   |

---

### 5.3 Markdown 處理函式

**def \_validate\_and\_fix\_markdown(markdown\_text)**

| 屬性   | 詳情                                                                                                                                         |
|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 位置   | phase5_postprocessing.py:180-220                                                                                                           |
| 風險等級 | 中風險 - Markdown 修復核心                                                                                                                        |
| 職責   | 驗證並修復常見的 Markdown 格式問題                                                                                                                     |
| 輸入   | markdown_text: str                                                                                                                         |
| 輸出   | str - 修復後的 Markdown                                                                                                                        |
| 關鍵邏輯 | 修復項目: <ul style="list-style-type: none"><li>1. Headers 前加換行</li><li>2. 未閉合的粗體標記</li><li>3. 表格格式化</li><li>4. 過多換行</li><li>5. 首尾空白</li></ul> |
| 修改影響 | <ul style="list-style-type: none"><li>• 影響最終顯示格式</li><li>• 不影響內容正確性</li></ul>                                                              |
| 修改場景 | <ul style="list-style-type: none"><li>• 新增額外的格式修復規則</li><li>• 調整修復策略</li></ul>                                                             |

---

### 5.4 品質檢查函式

**def \_quality\_check(response\_package)**

| 屬性   | 詳情                                             |
|------|------------------------------------------------|
| 位置   | phase5_postprocessing.py:253-321               |
| 風險等級 | 低風險 - 品質評估                                     |
| 職責   | 對最終回應進行品質評估                                    |
| 輸入   | response_package: Dict                         |
| 輸出   | Dict - 品質報告 (score, warnings, metrics, passed) |

---



| 屬性   | 詳情                                                                                                        |
|------|-----------------------------------------------------------------------------------------------------------|
| 關鍵邏輯 | 檢查項目:<br>1. 回應長度 (50-10000 字元)<br>2. Markdown 語法<br>3. 來源引用<br>4. Metadata 完整性<br>品質分數: 基線 100,每個警告 -10 分 |
| 修改影響 | <ul style="list-style-type: none"> <li>• 僅影響品質評估</li> <li>• 不影響實際輸出</li> </ul>                            |
| 修改場景 | <ul style="list-style-type: none"> <li>• 調整品質標準</li> <li>• 新增檢查項目</li> <li>• 修改評分公式</li> </ul>            |

## Phase 5 修改總結

最常見修改場景: 1. 新增 **Metadata** 欄位 → 修改 `_build_metadata()` 2. 調整品質標準 → 修改 `_quality_check()` 的檢查項目 3. 強化 **Markdown** 修復 → 調整 `_validate_and_fix_markdown()`

修改注意事項: - ✓ Phase 5 修改風險較低,大多是附加功能 - ✓ 品質檢查不影響實際輸出,可自由調整 - ⚠ Markdown 修復需確保不破壞原有格式

## 跨階段共享函式

### Cache 基礎架構

檔案: `libs/caching/streaming_cache.py`

**class StreamingCache**

| 屬性   | 詳情                                                                                                                      |
|------|-------------------------------------------------------------------------------------------------------------------------|
| 責任   | 提供 Redis 快取的統一介面                                                                                                        |
| 關鍵方法 | <code>get_async()</code> , <code>set_async()</code> , <code>get_phase_result()</code> , <code>set_phase_result()</code> |
| 使用階段 | Phase 1, 2, 4                                                                                                           |
| 修改風險 | 高風險 - 影響所有快取功能                                                                                                          |

### 資料庫查詢基礎

檔案: `libs/RAG/DB/AsyncDuckDBQuery.py`

**class AsyncDuckDBQuery**

| 屬性   | 詳情                                                                |
|------|-------------------------------------------------------------------|
| 責任   | 提供非同步 DuckDB 查詢介面                                                 |
| 關鍵方法 | <code>execute_async()</code> , <code>query_by_modeltypes()</code> |

| 屬性   | 詳情              |
|------|-----------------|
| 使用階段 | Phase 2         |
| 修改風險 | 高風險 - 影響結構化資料檢索 |

檔案: libs/RAG/DB/MilvusQuery.py

#### class MilvusQuery

| 屬性   | 詳情                  |
|------|---------------------|
| 責任   | 提供 Milvus 向量檢索介面    |
| 關鍵方法 | collection.search() |
| 使用階段 | Phase 2             |
| 修改風險 | 高風險 - 影響語義搜尋        |

## 修改影響分析矩陣

### 按修改類型分類

| 修改類型                       | 涉及階段          | 核心函式                                                   | 風險等級 | 測試範圍           |
|----------------------------|---------------|--------------------------------------------------------|------|----------------|
| 新增產品型號 <b>pattern</b>      | Phase 1       | _fast_path_extraction()                                | 低    | Phase 1 單元測試   |
| 調整 <b>Prompt template</b>  | Phase 1, 4    | QUERY_UNDERSTANDING_PROMPT, RESPONSE_GENERATION_PROMPT | 中    | 端到端測試          |
| 擴充 <b>ESSENTIAL_FIELDS</b> | Phase 2, 3    | _retrieve_from_duckdb(), truncate_context()            | 中    | Phase 2-4 整合測試 |
| 調整排序權重                     | Phase 3       | _rank_products_by_relevance()                          | 低    | Phase 3 單元測試   |
| 新增資料源                      | Phase 2       | _parallel_retrieve(), merge_results()                  | 高    | 完整回歸測試         |
| 修改快取策略                     | Phase 1, 2, 4 | _get_cached_*, cache_*                                 | 低    | 效能測試           |
| 調整 <b>Token</b> 限制         | Phase 3       | _truncate_context()                                    | 中    | Phase 3-4 整合測試 |
| 更換 <b>LLM</b> 模型           | Phase 1, 4    | _init_(), _llm_extraction(), process()                 | 高    | 完整回歸測試         |

### 按影響範圍分類

| 函式                             | 直接影響         | 間接影響         | 修改建議            |
|--------------------------------|--------------|--------------|-----------------|
| Phase1.process()               | Phase 1 流程   | Phase 2 輸入   | 謹慎修改,確保向後相容     |
| Phase1._fast_path_extraction() | Phase 1 快速路徑 | 無            | 可快速迭代           |
| Phase2.retrieve()              | Phase 2 流程   | Phase 3 輸入   | 謹慎修改            |
| Phase2._merge_results()        | 資料合併         | Phase 3 資料品質 | <b>關鍵:</b> 去重邏輯 |

| 函式                         | 直接影響      | 間接影響           | 修改建議    |
|----------------------------|-----------|----------------|---------|
| Phase3._truncate_context() | Token 管理  | Phase 4 LLM 輸入 | 影響資料完整性 |
| Phase4._build_prompt()     | Prompt 構建 | LLM 生成品質       | 需測試多種查詢 |
| Phase5._quality_check()    | 品質評估      | 無 (僅報告)        | 可自由調整   |

## 常見修改場景與最佳實踐

### 場景 1: 新增特徵識別 (如 "保固")

步驟: 1. 修改 Phase1.\_fast\_path\_extraction() 的 feature\_keywords 2. 修改 Phase3.\_truncate\_context() 的 feature\_field\_map 3. 確保 DuckDB 有對應欄位 4. 測試完整流程

風險: 低 - 向後相容

### 場景 2: 提升 LLM 回應準確度

步驟: 1. 分析當前回應問題 (缺少資料引用? 格式錯誤?) 2. 修改 Phase4.RESPONSE\_GENERATION\_PROMPT 3. 可選: 調整 Phase4.\_format\_product\_context() 格式 4. 多樣化測試 (比較/推薦/規格查詢)

風險: 中 - 需廣泛測試

### 場景 3: 新增 Elasticsearch 資料源

步驟: 1. 實作 Phase2.\_retrieve\_from\_elasticsearch() 2. 修改 Phase2.\_parallel\_retrieve() 加入第三個 task 3. 修改 Phase2.\_merge\_results() 處理三源合併 4. 調整 cache key 生成邏輯 5. 完整回歸測試

風險: 高 - 影響核心檢索邏輯

## 附錄: 快速修改檢查清單

### 修改前檢查

- [] 確認修改目的和預期效果
- [] 識別受影響的階段和函式
- [] 評估修改風險等級
- [] 準備測試資料和測試場景
- [] 備份原始程式碼

## 修改後檢查

- ☐ 執行相關的單元測試
- ☐ 執行整合測試 (受影響階段)
- ☐ 執行端到端測試 (高風險修改)
- ☐ 檢查 Log 輸出無異常
- ☐ 驗證效能沒有顯著下降
- ☐ 更新相關文檔

## 部署前檢查

- ☐ Code Review (中高風險修改必須)
- ☐ 完整回歸測試
- ☐ 效能測試 (如涉及資料庫或 LLM)
- ☐ 快取一致性測試 (如修改快取邏輯)
- ☐ 準備回滾計畫

---

文檔結束 | 版本 v1.0.0 | 2025-10-03